Assignment 9

Program –

1]

```
class Graph
   {

    constructor(noOfVertices)
    {
        this.noOfVertices = noOfVertices
        this.AdjList = new Map()
    }

  // add vertex to the graph
  addVertex(v)
  {
      this.AdjList.set(v, [])
  }
  // add edge to the graph
  addEdge(v, w)
  {
      //add an edge from v to w also
      this.AdjList.get(v).push(w)

      // add an edge from w to v also
      this.AdjList.get(w).push(v)
  }
// Prints the vertex and adjacency list
printGraph()
{
    // get all the vertices
    var get_keys = this.AdjList.keys()

    // iterate over the vertices
      for (var i of get_keys)
    {
          // great the corresponding adjacency list for the vertex
          var get_values = this.AdjList.get(i)
          var conc = ""

          // iterate over the adjacency list concatenate the values into
a string
          for (var j of get_values)
              conc += j + " ";

          // print the vertex and its adjacency list
          console.log(i + " -> " + conc);
      }
}

bfs(startingNode)
{
    // create a visited object
    var visited = {};
```

```
    // Create an object for queue
    var q = new Queue();

    // add the starting node to the queue
    visited[startingNode] = true;
    q.enqueue(startingNode);

    // loop until queue is element
    while (!q.isEmpty())
    {
        // get the element from the queue
        var getQueueElement = q.dequeue();

        // passing the current vertex to callback funtion
        console.log(getQueueElement);

        // get the adjacent list for current vertex
        var get_List = this.AdjList.get(getQueueElement);

        // loop through the list and add the element to the
        // queue if it is not processed yet
        for (var i in get_List) {
            var neighbour = get_List[i];

            if (!visited[neighbour]) {
                visited[neighbour] = true;
                q.enqueue(neighbour);
            }
        }
    }
}
dfs(startingNode)
{

    var visited = {};
    this.DFS_traversal(startingNode, visited);
}

// Recursive function which process and explore
// all the adjacent vertex of the vertex with which it is called
DFS_traversal(vert, visited)
{
    visited[vert] = true;
    console.log(vert);

    var get_neighbours = this.AdjList.get(vert);

    for (var i in get_neighbours) {
        var get_elem = get_neighbours[i];
        if (!visited[get_elem])
            this.DFS_traversal(get_elem, visited);
    }
}

}
class Queue
{
// Array is used to implement a Queue
```

```
constructor()
{
    this.items = [];
}

enqueue(element)
{
    // adding element to the queue
    this.items.push(element);
}
dequeue()
{

    if(this.isEmpty())
        return "Underflow";
    return this.items.shift();
}

isEmpty()
{
    // return true if the queue is empty.
    return this.items.length == 0;
}

}

var g = new Graph(6);
var vertices = ['A','B','C','D','E','F'];
for (var i = 0; i < vertices.length; i++)
{
g.addVertex(vertices[i]);
}

// adding edges
g.addEdge('A', 'B');
g.addEdge('A', 'D');
g.addEdge('A', 'E');
g.addEdge('B', 'C');
g.addEdge('D', 'E');
g.addEdge('E', 'F');
g.addEdge('E', 'C');
g.addEdge('C', 'F');
g.printGraph();
console.log("BFS");
g.bfs('A');
console.log("DFS");
g.dfs('A');

Output –
A -> B D E
B -> A C
C -> B E F
D -> A E
E -> A D F C
F -> E C
BFS
A
B
```

D
E
C
F
DFS
A
B
C
E
D
F